

US 1

Mise en place d'un environnement sécurisé

Projet : reverse engineering et investigation forensique

Date : 10 septembre 2026 | Statut : validée pour une exécution hors réseau

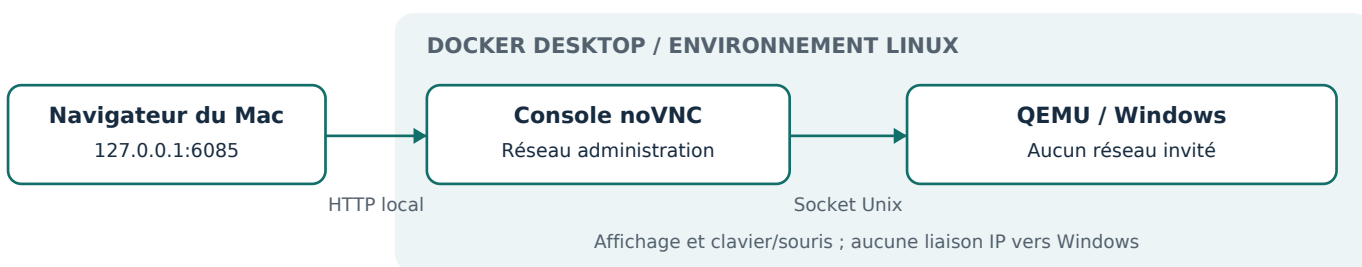
1. Objectif et périmètre

En tant qu'analyste malware, nous voulons exécuter le malware dans un environnement isolé afin de protéger le système hôte et d'observer son comportement.

Les deux critères du backlog sont : un environnement isolé (VM ou sandbox) et l'absence de fuite réseau non contrôlée. Ce rendu présente la mise en place du laboratoire et les vérifications associées. La reconnaissance, les imports de Res.exe et son comportement détaillé relèvent des US 2 à 4.

2. Environnement réalisé

La machine hôte est un Mac ARM64. Docker Desktop héberge un environnement Linux dans lequel QEMU émule un PC x64 exécutant Windows. QEMU fournit la machine invitée et son disque ; Docker organise les services et limite leurs accès.



Élément	Configuration retenue
Système invité	Windows 10 Enterprise LTSC Evaluation, x64, français
Processeur et mémoire	QEMU TCG, CPU qemu64, 2 processeurs virtuels, 4 Gio de RAM
Disque Windows	QCOW2 de 64 Gio de capacité virtuelle, volume Docker dédié
Service vm	QEMU et stockage Windows, sans réseau Docker ni carte réseau invitée
Service console	noVNC / websockify ; accès publié sur 127.0.0.1:6085
Accès graphique	Navigateur local vers la console, puis socket Unix VNC vers QEMU

Le chemin d'administration est : **navigateur du Mac** → **console noVNC** → **socket Unix** → **QEMU / Windows**. Le socket transporte l'affichage et les entrées clavier/souris ; il ne crée pas de carte réseau dans Windows.

La console possède son propre réseau Docker d'administration. Le service vm n'y est pas connecté. Aucun port TCP de QEMU n'est publié.

Les fichiers utiles se trouvent dans infrastructure/windows : compose.yaml définit les deux services ; Dockerfile construit leurs images ; download_windows.py récupère l'ISO ; install.sh réalise la première installation ; start.sh démarre l'installation existante.

3. Mesures d'isolation

Mesure appliquée	Effet recherché
QEMU : -nic none	Aucun périphérique réseau virtuel fourni à Windows. [1]
Docker : network_mode: none pour vm	Aucune interface externe dans le conteneur de la VM. [2]
Console : 127.0.0.1:6085:6080	Publication sur l'interface de boucle locale du Mac. [3]
Utilisateur 1000:1000, cap_drop: [ALL], no-new-privileges:true	Réduction des privilèges des services.
Racine du conteneur en lecture seule	Écritures limitées aux volumes et aux espaces temporaires prévus.
ISO en lecture seule ; aucun partage de dossier invité ni USB hôte	Transfert par DVD virtuel sans dossier hôte accessible en écriture depuis Windows.

Le disque Windows reste volontairement inscriptible afin d'observer les changements produits dans la VM. Le dossier hôte media est monté en lecture seule dans QEMU et exposé à Windows au moyen d'une image de DVD. Le socket Docker de l'hôte n'est pas monté.

4. Vérifications et critères d'acceptation

Les contrôles ci-dessous distinguent les observations faites dans Windows pendant la séance et les contrôles refaits pour ce rendu. Les sorties brutes de cette seconde série sont conservées dans preuves.txt. Windows n'a pas été redémarré pour rédiger le document.

Contrôle	Résultat observé et provenance
Interfaces Windows : ipconfig /all	Aucun adaptateur réseau externe utilisable. Observation de séance du 10/09.
Routes Windows : route print	Boucle locale uniquement ; aucune route par défaut IPv4 ou IPv6. Observation de séance.
Sortie Windows : ping 1.1.1.1	Échec de transmission (« défaillance générale »). Observation de séance.
Sortie du conteneur vm vers 1.1.1.1:443	Network is unreachable, code 1 attendu. Test refait le 10/09 dans un conteneur temporaire du même service, sans démarrer Windows.
Paramètres réels des conteneurs	network=none, -nic none, privilèges restreints et console liée à 127.0.0.1 confirmés par docker inspect.

Critère 1 - Environnement isolé : validé. Le programme est exécuté dans un système Windows invité, avec un disque dédié et les accès décrits ci-dessus.

Critère 2 - Absence de fuite réseau non contrôlée : validé dans cette configuration hors réseau. L'absence d'interface et de route dans Windows est corroborée par l'isolation du conteneur. L'échec d'un ping seul ne constituerait pas cette preuve.

5. Retour à un état propre

Le snapshot disque windows-clean-us1 a été créé avant l'introduction du malware. Pendant la séance, sa restauration a été testée sur une copie du disque : une modification volontaire rendait la copie différente, puis l'application du snapshot rétablissait un contenu identique à la référence.

Après l'essai ultérieur, l'état du disque a été conservé dans malware-first-run, puis windows-clean-us1 a été réappliqué avec succès. Les deux snapshots ont été retrouvés lors de la vérification du rendu. La VM est arrêtée proprement, avec un code de sortie 0.

6. Démonstration de l'US 1

Prévoir environ trois minutes dans la démonstration du projet. Les commandes ci-dessous partent de la racine du dépôt ; Docker Desktop doit être lancé.

Démarrer la VM existante :

```
bash infrastructure/windows/start.sh
```

Ouvrir ensuite la console Windows et déverrouiller la session.

Montrer l'absence de réseau depuis Windows :

```
ipconfig /all
route print
ping 1.1.1.1
```

Les résultats attendus sont ceux du tableau de validation : aucune interface externe, aucune route par défaut et échec de la transmission.

Montrer les protections Docker depuis le Mac :

```
docker inspect cyber-windows-vm-1 --format \
'{{.HostConfig.NetworkMode}} {{.HostConfig.Privileged}}'
docker port cyber-windows-console-1
```

Résultats attendus : none false et 6080/tcp -> 127.0.0.1:6085.

Arrêter Windows puis montrer le point de restauration :

Depuis Windows : shutdown /s /t 0. Attendre son arrêt complet. Depuis le Mac :

```
docker wait cyber-windows-vm-1
docker compose -f infrastructure/windows/compose.yaml \
run --rm --no-deps vm \
qemu-img snapshot -l /storage/windows.qcow2
```

Le code de sortie attendu est 0. Le snapshot windows-clean-us1 doit apparaître. Si une restauration est nécessaire après conservation des résultats, remplacer la dernière ligne par `qemu-img snapshot -a windows-clean-us1 /storage/windows.qcow2`, toujours avec Windows arrêté.

Utiliser start.sh pour les séances suivantes. Le script install.sh recrée le disque Windows : il est réservé à la première installation.

7. Limites et état livré

Le mode hors réseau empêche d'observer une communication aboutie avec un serveur distant. Une étude du trafic nécessiterait ultérieurement un réseau de laboratoire contrôlé, sans sortie Internet. L'isolation décrite réduit l'exposition ; aucune résistance à une évocation de VM n'a été testée.

L'émulation x64 sur ARM64 peut ralentir Windows et modifier le comportement temporel observé. Windows affiche une licence d'évaluation expirée : l'activation n'est pas validée. Le rendu concerne l'environnement Windows ; l'infrastructure Linux reste à réaliser.

À la livraison, le disque actif a été restauré à l'état propre et la VM reste arrêtée. La console locale reste disponible. Les snapshots et captures de travail restent dans les volumes Docker locaux ; ils ne sont pas inclus dans le rendu.

Références et pièces du rendu

- Backlog fourni : Backlog - Reverse engineering.pdf, page 5 (livrables) et page 7 (US 1).
- Configuration du laboratoire : compose.yaml et Dockerfile.
- Vérifications Docker du 10/09/2026 : preuves.txt. Les constats Windows et le test de restauration sur copie sont des observations de séance, pas de nouvelles captures jointes.
- [1] QEMU - Invocation, options réseau.
- [2] Docker - None network driver.
- [3] Docker - Publication des ports.